

Infrastructure as code, and end-to-end automation

The part that turns “it works on the cloud” into “it rebuilds itself from nothing”.

Dr. Abhijith Anandakrishnan · Assistant Professor
24AIM332 · Odd Semester 2026-27

If your infrastructure exists because someone clicked, it does not really exist.

You have a machine. You do not have a way to get another one.

Clicking vs declaring

● Console clicking

- Nobody knows what was done
- No review, no history
- Staging drifts from production
- Rebuilding = archaeology
- The person who knows leaves

● Infrastructure as code

- The repo **is** the infrastructure
- Reviewed like any change
- Environments differ by variables
- Rebuild = one command
- New joiner reads the repo

What makes IaC work

Declarative — you describe the end state, not the steps.

Idempotent — applying twice changes nothing the second time.

Versioned — infrastructure history is git history.

Idempotence is the one that matters most in practice. It means "apply" is always safe to run, which means you run it often, which means drift never accumulates.

The concept that confuses everyone

The tool records what it created in a **state file**, mapping your declarations to real resource IDs.

```
plan = diff(desired, state, reality)
```

State must be ****shared and locked**** for a team: remote backend, not a laptop. Two people applying against two local state files will fight over the same resources, and one will silently destroy the other's work.

THE LOOP

plan, read, apply

```
tofu plan    # what WOULD change. Read this. Every time.  
tofu apply   # make it so  
tofu destroy # remove it, and stop billing
```

A PLAN SHOWING “DESTROY” ON A DATABASE IS HOW YOU LEARN TO READ PLANS

Provisioning vs configuration

Provisioning — bring resources into existence.

Servers, networks, firewalls, DNS, object storage.

OpenTofu, Terraform, Bicep, CloudFormation

Configuration — make an existing machine correct.

Packages, users, services, files, certificates.

Ansible, Chef, Puppet

An environment should be a set of variables, not a set of scripts.

Staging and production run the same code with different values. Otherwise staging proves nothing.

PART TWO

CI/CD

CI and CD are different promises

Continuous integration

Every push is built and tested, automatically, on a clean machine.

Answers: *does it work anywhere other than your laptop?*

Continuous delivery / deployment

Every change that passes goes to an environment, automatically.

Answers: *how long from merge to running?*

What runs, in order

```
push
├─ lint          seconds
├─ unit tests    a minute
├─ build image   minutes
├─ integration   minutes
├─ deploy staging
├─ smoke tests
└─ deploy prod   (gate)
```

Fast things first. A lint error should fail in 20 seconds, not after a 9-minute build.

Each stage is a **gate**: nothing downstream runs if it fails.

Four rules that make a pipeline trustworthy

1. Build once, promote the artefact. Rebuilding per environment means you tested something other than what you shipped.

2. No secrets in the repo. Injected at run time from a secret store.

3. Reproducible. Pinned versions, clean runner. “Works on the third rerun” is a failure.

4. Fast. A pipeline over ten minutes stops being a feedback loop and becomes a queue.

A minimal pipeline

```
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make lint
      - run: make test

  deploy:
    needs: test # gate
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make build push
      - run: make deploy ENV=staging
```


End to end: a commit becomes running infrastructure with no human step.

That is the whole ambition, and it is achievable for a student project.

From `git push` to a URL

1. Push to a branch
2. CI lints, tests, builds an image
3. Image pushed to a registry, tagged by commit
4. `tofu plan` posted to the pull request
5. Merge
6. `tofu apply` creates or updates infrastructure
7. Config management brings hosts to spec
8. Smoke test proves it answers
9. A URL works

Should merging actually build the infrastructure?

- **Plan only**

Cheap, safe, and proves nothing.

You discover the template is broken on the day you need it.

- **Apply, assert, destroy**

Build it for real, assert it converged, tear it down in the same run.

Costs a few rupees. Buys the knowledge that it works.

Leave it standing only when the run fails.

That is the one case worth paying to inspect.

WHERE THIS IS GOING

Two real projects, next

Echno — construction intelligence platform

OpenTofu + Ansible + Kubernetes, three environments, one set of roles.

A digital twin on Azure

Bicep, managed certificates, observability, and a pipeline that builds and destroys infrastructure on every merge.

Not diagrams from a textbook.

Systems that are running, with the mistakes left in.